

Finite Element Assembly

One-dimensional linear elements

What assembly must accomplish

For the axial bar,

$$\int_0^L AE u_h' v_h' dx = \int_0^L f v_h dx + \bar{P} v_h(L).$$

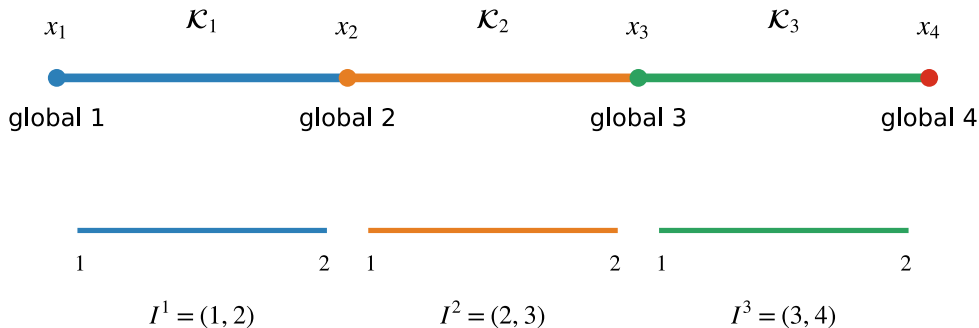
Each element supplies a small matrix and vector,

$$\mathbf{K}^e = \int_{\mathcal{K}_e} (\mathbf{B}^e)^T AE \mathbf{B}^e dx, \quad \mathbf{F}^e = \int_{\mathcal{K}_e} (\mathbf{N}^e)^T f dx.$$

Assembly must place these local entries into the correct rows and columns of

$$\mathbf{KU} = \mathbf{F}.$$

Local and global node numbers



Local node numbers restart on each element. Global node numbers identify shared nodes of the complete mesh.

The three maps used in assembly

For a two-node element,

$$I^e : \{1, 2\} \longrightarrow \{1, \dots, N + 1\}, \quad I^e(1) = e, \quad I^e(2) = e + 1.$$

For a global node that belongs to element e ,

$$J^e : \{I^e(1), I^e(2)\} \longrightarrow \{1, 2\}, \quad J^e(I^e(a)) = a.$$

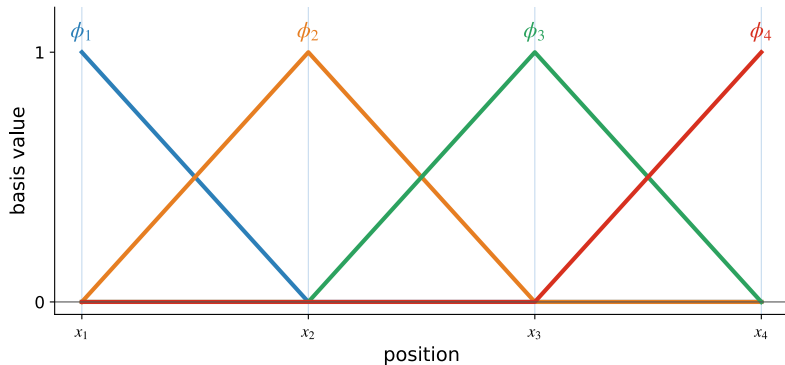
The elements incident on global node i are

$$\mathcal{E}(i) = \{e : x_i \in \overline{\mathcal{K}}_e\}.$$

For the shared node 2,

$$J^1(2) = 2, \quad J^2(2) = 1, \quad \mathcal{E}(2) = \{1, 2\}.$$

Local support determines the interacting basis functions



$$\text{supp } \phi_i = \bigcup_{e \in \mathcal{E}(i)} \bar{\mathcal{K}}_e.$$

On element e , only the basis functions associated with its two nodes are nonzero.

A global function restricted to one element

The global finite element function is

$$u_h(x) = \sum_{i=1}^{N+1} U_i \phi_i(x).$$

On $\mathcal{K}_e$, all terms except those associated with its two nodes vanish:

$$u_h|_{\mathcal{K}_e} = U_{I^e(1)} \phi_1^e + U_{I^e(2)} \phi_2^e = \mathbf{N}^e \mathbf{U}^e,$$

where

$$\mathbf{U}^e = \begin{bmatrix} U_{I^e(1)} \\ U_{I^e(2)} \end{bmatrix}.$$

Connectivity extracts the local coefficient vector from the global vector.

Load-vector assembly by global entry

The distributed part of global entry i is

$$F_i^{\text{dist}} = \sum_{e=1}^N \int_{\mathcal{K}_e} f \phi_i \, dx.$$

Only elements in $\mathcal{E}(i)$ contribute. Define

$$F_a^e = \int_{\mathcal{K}_e} f \phi_a^e \, dx.$$

Since $\phi_i|_{\mathcal{K}_e} = \phi_{J^e(i)}^e$,

$$F_i^{\text{dist}} = \sum_{e \in \mathcal{E}(i)} F_{J^e(i)}^e.$$

This formula fixes a global node i and gathers all incident-element contributions.

Three-element load vector

For

$$I^1 = (1, 2), \quad I^2 = (2, 3), \quad I^3 = (3, 4),$$

the distributed-load vector is

$$\mathbf{F}^{\text{dist}} = \begin{bmatrix} F_1^1 \\ F_2^1 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ F_1^2 \\ F_2^2 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ F_1^3 \\ F_2^3 \end{bmatrix} = \begin{bmatrix} F_1^1 \\ F_2^1 + F_1^2 \\ F_2^2 + F_1^3 \\ F_2^3 \end{bmatrix}.$$

At a shared node, the neighboring element contributions add.

Stiffness-matrix assembly by global entry

The global entry is

$$K_{ij} = \sum_{e=1}^N \int_{\mathcal{K}_e} AE \phi_j' \phi_i' dx.$$

An element contributes only if it contains both nodes i and j . With

$$K_{ab}^e = \int_{\mathcal{K}_e} AE (\phi_b^e)' (\phi_a^e)' dx,$$

we obtain

$$K_{ij} = \sum_{e \in \mathcal{E}(i) \cap \mathcal{E}(j)} K_{J^e(i), J^e(j)}^e.$$

For example,

$$K_{22} = K_{22}^1 + K_{11}^2, \quad K_{23} = K_{12}^2, \quad K_{14} = 0.$$

Three-element stiffness matrix

$$\mathcal{K}_1 : I^1 = (1, 2)$$

K_{11}^1	K_{12}^1
K_{21}^1	K_{22}^1

$$\mathcal{K}_2 : I^2 = (2, 3)$$

K_{11}^2	K_{12}^2
K_{21}^2	K_{22}^2

$$\mathcal{K}_3 : I^3 = (3, 4)$$

K_{11}^3	K_{12}^3
K_{21}^3	K_{22}^3



K_{11}^1	K_{12}^1	0	0
K_{21}^1	$K_{22}^1 + K_{11}^2$	K_{12}^2	0
0	K_{21}^2	$K_{22}^2 + K_{11}^3$	K_{12}^3
0	0	K_{21}^3	K_{22}^3

shared entries accumulate contributions from adjacent elements

The assembly operation used in code

The entry formulas gather contributions for a fixed global index. A program usually loops over elements and scatters local entries:

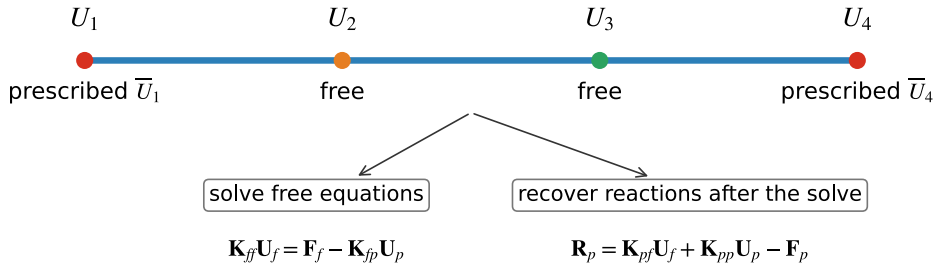
$$F_{I^e(a)} += F_a^e, \quad K_{I^e(a), I^e(b)} += K_{ab}^e.$$

```
for e in range(num_elements):
    dofs = connectivity[e]
    K_e, F_e = element_arrays(e)

    for a in range(2):
        F[dofs[a]] += F_e[a]
        for b in range(2):
            K[dofs[a], dofs[b]] += K_e[a, b]
```

The arrays use zero-based Python indices. The mathematical map I^e uses one-based indices.

Essential boundary conditions



With free indices f and prescribed indices p ,

$$\mathbf{K}_{ff}\mathbf{U}_f = \mathbf{F}_f - \mathbf{K}_{fp}\mathbf{U}_p.$$

Solve only for the free coefficients, then insert the prescribed values into the complete vector $\mathbf{U}$.

Reactions and final checks

After constructing the complete coefficient vector,

$$\mathbf{R} = \mathbf{KU} - \mathbf{F}.$$

The entries associated with prescribed degrees of freedom are the reactions:

$$\mathbf{R}_p = \mathbf{K}_{pf} \mathbf{U}_f + \mathbf{K}_{pp} \mathbf{U}_p - \mathbf{F}_p.$$

Before accepting the solution, check:

- ▶ the connectivity and element lengths;
- ▶ the free and prescribed degree-of-freedom sets;
- ▶ global equilibrium using applied loads and reactions;
- ▶ displacement and axial-force trends against the loading.

Complete finite element procedure

1. Create node coordinates and element connectivity.
2. Initialize the global matrix $\mathbf{K}$ and vector $\mathbf{F}$.
3. For each element, compute $\mathbf{K}^e$ and $\mathbf{F}^e$.
4. Scatter-add the element entries using $\mathbf{l}^e$.
5. Separate free and prescribed degrees of freedom.
6. Solve the reduced system for $\mathbf{U}_f$.
7. Insert prescribed values and compute reactions.
8. Evaluate displacement, strain, stress, and axial force elementwise.

Notebook demonstration problem

Set $L = AE = f_0 = 1$ and solve

$$-u'' = 1 \quad \text{in } (0, 1), \quad u(0) = 0, \quad u'(1) = 0.$$

The exact solution is

$$u(x) = x - \frac{x^2}{2}.$$

During the demonstration:

- ▶ assemble and inspect $\mathbf{K}$ and $\mathbf{F}$ for one mesh;
- ▶ impose the essential boundary condition and solve;
- ▶ examine the displacement error; and
- ▶ then introduce derivative-based errors and mesh refinement.

Element loop in the demonstration

For uniform linear elements with $h = x_{e+1} - x_e$,

$$\mathbf{K}^e = \frac{1}{h} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad \mathbf{F}^e = \frac{h}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

```
for e in range(num_elements):
    dofs = np.array([e, e + 1])
    h_e = nodes[e + 1] - nodes[e]

    K_e = np.array([[1.0, -1.0],
                    [-1.0, 1.0]]) / h_e
    F_e = 0.5 * h_e * np.ones(2)

    for a in range(2):
        F[dofs[a]] += F_e[a]
        for b in range(2):
            K[dofs[a], dofs[b]] += K_e[a, b]
```

Solve, recover the reaction, and compare

```
num_nodes = len(F)
U = np.zeros(num_nodes)
K_ff = K[1:, 1:]
F_f = F[1:]
U[1:] = np.linalg.solve(K_ff, F_f)

reaction = (K @ U - F)[0]
```

For this problem, the computed nodal values agree with the exact quadratic solution at the nodes. Between nodes, u_h is piecewise linear. Mesh refinement reduces the interpolation error.

Begin with the displacement error

Let $e = u - u_h$. We first examine errors in the solution values:

$$\|e\|_{L^2} = \left(\int_0^1 e^2 dx \right)^{1/2}, \quad \|e\|_{\infty} = \max_{x \in [0,1]} |e(x)|.$$

For this problem, u_h agrees with u at the nodes. The exact solution is quadratic, while u_h is linear on each element. The error therefore vanishes at the nodes but is nonzero inside each element.

For linear elements in this particular problem,

$$\|e\|_{L^2} = O(h^2), \quad \|e\|_{\infty} = O(h^2).$$

Energy and H^1 errors

The preceding norms do not measure the derivative error. For the bar problem,

$$\|e\|_E = \left(\int_0^1 AE(e')^2 dx \right)^{1/2},$$

while

$$\|e\|_{H^1} = (\|e\|_{L^2}^2 + \|e'\|_{L^2}^2)^{1/2}.$$

Here $AE = 1$, so the energy norm is the H^1 seminorm. Since $e(0) = 0$, the energy and H^1 norms are equivalent by the Poincaré inequality.

For linear elements and this smooth solution,

$$\|e\|_E = O(h), \quad \|e\|_{H^1} = O(h).$$

Analysis in the notebook

For one mesh, the notebook displays:

- ▶ the finite element and exact solutions in the left panel;
- ▶ the pointwise absolute error in the right panel; and
- ▶ the L^2 and supremum errors in the error-plot title.

Only after interpreting these errors do we compute the energy and H^1 errors. The mesh-refinement loop then displays:

- ▶ all finite element solutions with the exact solution; and
- ▶ all four error norms against h on log-log axes.

For this problem, the fitted slopes should be approximately 2 for the L^2 and supremum errors and approximately 1 for the energy and H^1 errors.